

SESSION 3

Neural Networks & Backprop, from scratch

What I learned building Karpathy's micrograd — a tiny autograd engine that trains a neural net in ~150 lines.



forward → data flows in • ← backward gradients flow out

What is micrograd?

- A scalar-valued automatic gradient engine (autograd).
- It builds a math expression as a graph, then computes the derivative of the output w.r.t. every input automatically.
- That derivative-of-everything is exactly what training a neural network needs.
- Works on single numbers, not tensors — purely for teaching. None of the math is lost.
- It's the same core algorithm inside PyTorch / TensorFlow, just without the GPU + tensor engineering.

THE WHOLE LOOP

1 forward push inputs through the graph → a prediction

2 loss one number: how wrong the prediction is

3 backward chain rule fills in every gradient

4 update nudge each weight opposite its gradient

The path we'll walk

01

Derivatives

nudge an input, watch the output

02

The Value object

wrap a scalar, remember its history

03

Computation graph

expressions become a DAG

04

Chain rule

the one idea backprop rests on

05

Automatic backward

each op knows its local derivative

06

Neuron / Layer / MLP

build the network from Values

07

Loss + training loop

forward, backward, update, repeat

08

vs. the real world

what PyTorch adds on top

A derivative is just a nudge

The slope answers one question:

if I nudge the input by a tiny h , how much does the output move — and in which direction?

$$\frac{df}{dx} = \lim_{h \rightarrow 0} [f(x+h) - f(x)] / h$$

- Positive slope $\rightarrow$ push x up to raise the output.
- Negative slope $\rightarrow$ push x down to raise it.
- **A gradient is just this, bundled for many inputs at once.**

```
# nudge one input, measure the response
```

```
def f(a, b, c):  
    return a*b + c
```

```
h = 0.0001  
base = f(2.0, -3.0, 10.0)  
bumped = f(2.0 + h, -3.0, 10.0)
```

```
slope = (bumped - base) / h  
#  $\approx -3.0$   $\leftarrow$  that's  $\partial f / \partial a$ 
```

The Value object

```
class Value:
    def __init__(self, data, _children=(), _op=''):
        self.data = data          # the number (forward)
        self.grad = 0.0          #  $\partial L / \partial \text{self}$  (backward)
        self._prev = set(_children) # who made me
        self._op = _op           # which operation
        self._backward = lambda: None

    # operator overloads build the graph as you compute
    def __add__(self, other): ...
    def __mul__(self, other): ...
    def tanh(self): ...
```

EVERY node carries

data
0.50
 ∇ 0.13

data the value itself — filled during the forward pass.

grad how much this node affects the final loss — filled during backward.

Same node, two numbers. This split trips everyone up at first.

Expressions become a graph

```
a = Value(2.0)
b = Value(-3.0)
c = Value(10.0)
```

```
e = a * b    # -6.0
d = e + c    # 4.0
```



- Each operation creates a new node...
- ...that remembers its inputs (`_prev`).
- **The result is a DAG we can differentiate.**

$$d = a \cdot b + c$$

The chain rule is the whole engine

$$\partial L / \partial x = \partial L / \partial y \times \partial y / \partial x$$

gradient at x = gradient flowing in from above × local derivative of this op

+ addition

out = a + b

$\partial \text{out} / \partial a = 1$

passes the gradient straight through

× multiply

out = a · b

$\partial \text{out} / \partial a = b$

routes gradient, scaled by the other input

tanh

out = tanh(x)

$\partial \text{out} / \partial x = 1 - \text{out}^2$

squashes; slope shrinks near ±1

Backprop by hand



■ data — forward

■ grad — backward

- **Seed: d.grad = 1 ($\partial L / \partial L = 1$)**
- + node: pass 1 through $\rightarrow$ c.grad = 1, e.grad = 1
- $\times$ node: swap operands $\rightarrow$ a.grad = b = -3, b.grad = a = 2
- Read it off: nudging a by $+\epsilon$ changes the output by -3ϵ .
- The 1 is the **START** of the backward pass — not the value of the output.

One neuron = one small graph

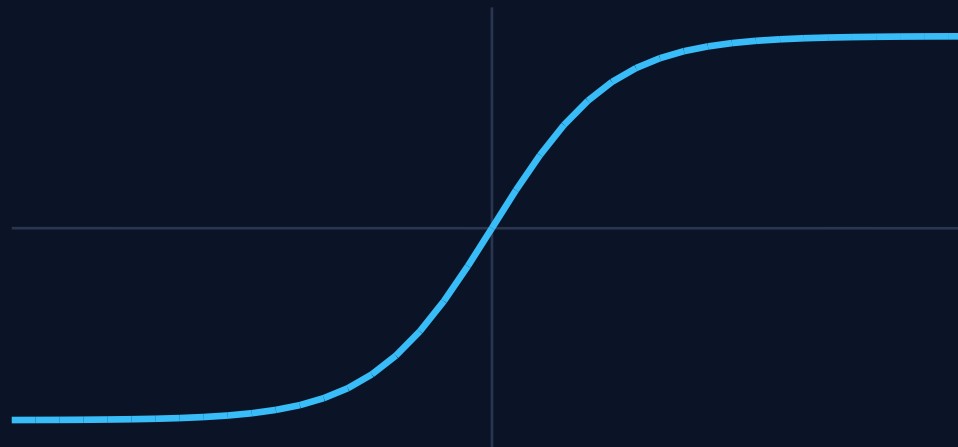
```
# a neuron is just a weighted sum
# squashed by a nonlinearity

n = tanh( w0 · x0 + w1 · x1 + b )

# w = weights (learned)
# b = bias      (learned)
# tanh = the squashing nonlinearity
```

- Without the nonlinearity, stacked layers collapse into one linear map.
- tanh keeps outputs in $(-1, 1)$ and gives a clean derivative: $1 - n^2$.

tanh(x)



slope $\rightarrow 0$ near the tails • steepest at 0

Teach each op to back itself up

```
def __mul__(self, other):
    out = Value(self.data * other.data,
                (self, other), '*')

    def _backward():
        self.grad += other.data * out.grad
        other.grad += self.data * out.grad
    out._backward = _backward
    return out

# local derivative (b, a) × incoming grad
# ...the chain rule, stored right on the node
```

- Each operation attaches a `_backward` closure.
- It knows only its own local derivative...
- ...and multiplies it by `out.grad` flowing in.
- `+` → grad passes through
- `×` → grad × the other operand
- `tanh` → grad × (1 - out²)
- **No node needs the global picture — local rules compose into full backprop.**

backward(): order matters

```
def backward(self):
    topo, seen = [], set()
    def build(v):
        if v not in seen:
            seen.add(v)
            for child in v._prev: build(child)
            topo.append(v)
    build(self)

    self.grad = 1.0          # seed  $\partial L / \partial L$ 
    for v in reversed(topo):
        v._backward()        # push grads back
```

- A node can only send its gradient once it has received all of its own.
- So sort the graph topologically...
- **...seed the output at 1.0...**
- ...then call `_backward()` in reverse order.
- **One `.backward()` on the loss fills in every gradient in the whole network.**

The bug everyone hits: +=

When a node feeds MORE THAN ONE place...

$$b = a + a$$

$db/da = 2$, not 1. The gradient arrives along BOTH edges and must add up.

```
self.grad = ...    # WRONG
# overwrites earlier contributions

self.grad += ...    # RIGHT
# accumulates every path to the loss
```

- **Multivariable chain rule: contributions from every path SUM.**
- Using `=` keeps only the last edge's gradient — silently wrong.
- Using `+=` is why grads start at 0.0 and accumulate.
- **This exact bug returns in the training loop: forget to reset grads and they pile up across steps.**
- Fix there: zero the grads before every backward pass.

Two sanity checks

tanh doesn't have to be atomic

```
# break tanh into smaller ops:  
e = (2*x).exp()  
t = (e - 1) / (e + 1)
```

Backprop still gives the identical gradient. You can implement at any granularity — only correct local derivatives matter.

...and it matches PyTorch

```
import torch  
x = torch.tensor([2.0],  
                 requires_grad=True)  
y = (x*w + b).tanh()  
y.backward()  
x.grad  # same number micrograd gives
```

Same graph, same gradients. micrograd $\approx$ scalar PyTorch — the concepts transfer directly.

Neuron → Layer → MLP

Neuron

```
class Neuron:
    w=[Value(..) ..]
    b= Value(..)
    def __call__(x):
        act=sum(
            wi*xi)+b
        return act.tanh()
```

Layer

```
class Layer:
    # list of
    # neurons
    neurons=[
        Neuron(n)
        for _ ..]
    def __call__:
        [n(x) ..]
```

MLP

```
class MLP:
    # stack of
    # layers
    def __call__(x):
        for L in layers:
            x=L(x)
        return x
```

2 → 3 → 2 → 1

A tiny composition:
an MLP is layers,
a layer is neurons,
a neuron is Values.

So the whole net is
one big graph —
backprop just works.

- Because every parameter is a Value, calling `.backward()` on the loss reaches all of them — no special network code needed.

Loss: one number for 'how wrong'

$$L = \sum (\hat{y}_i - y_i)^2$$

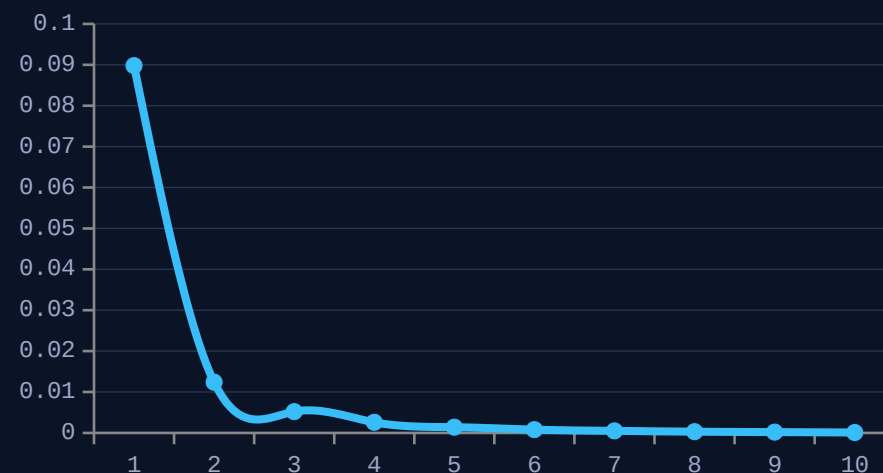
- Compare prediction $\hat{y}$ to the target y .
- Square the gap $\rightarrow$ always positive, punishes big misses.
- Sum over examples $\rightarrow$ a single scalar.
- **Low loss = the network is right. That's the goal.**

- **The loss just adds a few nodes onto the END of the same graph.**
- It's still one connected DAG: inputs $\rightarrow$ neurons $\rightarrow \hat{y} \rightarrow$ loss.
- So backprop from L flows all the way back to every weight.
- Seed L.grad = 1 — the value of L (e.g. 0.09) is separate.

The training loop

```
for step in range(20):  
  
    ypred = [model(x) for x in xs] # 1 forward  
    loss = sum((yp-yt)**2  
               for yp,yt in ...)  
  
    for p in model.parameters():  
        p.grad = 0.0 # zero_grad!  
    loss.backward() # 2 backward  
  
    for p in model.parameters():  
        p.data += -0.05 * p.grad # 3 update
```

LOSS PER STEP



- zero → backward → nudge, repeat.
- Loss falls 0.09 → ~0.0001 as $\hat{y} \rightarrow$ target.
- backward computes; the update USES the grads.

micrograd vs. the real world

micrograd	PyTorch / TensorFlow	what actually changes
scalars	tensors (arrays on GPUs)	same math, batched for speed
~5 operations	hundreds of ops	conv, attention, norm — all just nodes
manual SGD	Adam, schedulers	smarter use of the same gradients
~150 lines	millions of lines	engineering around one core idea

Forward → build a graph → backward (chain rule) → update. Everything else is built on top.

FIVE THINGS TO REMEMBER

- 1 data vs grad**

Every node holds its value and, separately, $\partial L / \partial \text{node}$.
- 2 chain rule**

Local derivative $\times$ incoming gradient — that's all backprop is.
- 3 seed with 1**

Backward starts at $L.\text{grad} = 1$, not the loss value.
- 4 one graph**

Loss and network are one DAG; `.backward()` reaches every weight.
- 5 compute $\neq$ update**

Backward computes gradients; the update step uses them to cut the loss.

~150 lines contain the entire core of how neural networks learn.